

Higher-Order Functions

Example 1 : Apply Function

A simple higher-order function that applies a function to a value:

$[X, Y]$
$apply : (X \rightarrow Y) \rightarrow (X \rightarrow Y)$
$\forall f : X \rightarrow Y; x : X \bullet apply(f)(x) = f(x)$

`apply` takes a function and a value, returns the result.

Example 2 : Using Apply

$square : \mathbb{N} \rightarrow \mathbb{N}$ $result : \mathbb{N}$
$\forall n : \mathbb{N} \bullet square(n) = n * n$ $result = apply(square)(5)$

`result` = 25

Example 3 : Compose Function

Function composition as a higher-order operation:

$[A, B, C]$
$compose : (B \rightarrow C) \times (A \rightarrow B) \rightarrow (A \rightarrow C)$
$\forall f : B \rightarrow C; g : A \rightarrow B; x : A \bullet compose(f, g)(x) = f(g(x))$

`compose` takes two functions and returns their composition.

Example 4 : Using Compose

$addOne : \mathbb{N} \rightarrow \mathbb{N}$ $double : \mathbb{N} \rightarrow \mathbb{N}$ $addOneAndDouble : \mathbb{N} \rightarrow \mathbb{N}$
$\forall n : \mathbb{N} \bullet addOne(n) = n + 1$ $\forall n : \mathbb{N} \bullet double(n) = 2 * n$ $addOneAndDouble = compose(double, addOne)$

`addOneAndDouble(5)` = `double(addOne(5))` = `double(6)` = 12

Example 5 : Twice Function

A function that applies another function twice:

$[X]$
$twice : (X \rightarrow X) \rightarrow (X \rightarrow X)$
$\forall f : X \rightarrow X; x : X \bullet twice(f)(x) = f(f(x))$

`twice` takes a function and returns a function that applies it twice.

Example 6 : Using Twice

$increment : \mathbb{N} \rightarrow \mathbb{N}$
$addTwo : \mathbb{N} \rightarrow \mathbb{N}$
$\forall n : \mathbb{N} \bullet increment(n) = n + 1$
$addTwo = twice(increment)$

$addTwo(5) = increment(increment(5)) = 7$

Example 7 : Constant Function

A higher-order function that returns a constant function:

$[X, Y]$
$constant : Y \rightarrow (X \rightarrow Y)$
$\forall y : Y; x : X \bullet constant(y)(x) = y$

constant takes a value and returns a function that always returns that value.

Example 8 : Using Lambda with Higher - Order Functions

$resultLambda : \mathbb{N}$
$resultLambda = apply((\lambda x : \mathbb{N} \bullet x * x))(7)$

$resultLambda = 49$

Example 9 : Flip Function

A function that flips the arguments of a binary function:

$[X, Y, Z]$
$flip : (X \times Y \rightarrow Z) \rightarrow (Y \times X \rightarrow Z)$
$\forall f : X \times Y \rightarrow Z; x : X; y : Y \bullet flip(f)(y, x) = f(x, y)$

flip swaps the order of arguments to a function.

Example 10 : Best Practices

When using higher-order functions: 1. Use type parameters to make functions generic 2. Keep function signatures clear and simple 3. Compose small functions to build complex behavior 4. Use lambda expressions for inline function definitions 5. Higher-order functions enable functional programming patterns